



Procesadores del Lenguaje
Grado en Ingeniería Informática

Desarrollo de un Traductor de C a Common Lisp

Memoria Técnica: Gestión de Ámbitos, Estructuras de Control
y Funciones

Miembros del grupo:

Nombre del Alumno 1

Nombre del Alumno 2

Grupo XX - Equipo YY

2025/2026

1. Introducción

El presente proyecto consiste en el diseño y construcción de un compilador traductor que transforma código fuente escrito en un subconjunto del lenguaje C hacia código ejecutable en Common Lisp. Este proceso implica no solo un cambio de sintaxis, sino una adaptación de paradigmas: del modelo imperativo de C al modelo basado en expresiones y listas de Lisp.

A lo largo de esta memoria se detallan los retos técnicos superados, las correcciones gramaticales realizadas para garantizar la compatibilidad con el código de prueba y la lógica de implementación de los componentes más complejos del lenguaje.

2. Desarrollo y Decisiones de Diseño

2.1. Gestión de Ámbitos y Variables Locales

Uno de los mayores desafíos fue emular el comportamiento de las variables locales de C. En Lisp, el uso de `setq` define variables en el entorno global. Para evitar que una variable `temp` en la función suma sobrescribiera una variable `temp` en `main`, adoptamos una estrategia de **prefijado por función**.

Cada vez que el analizador detecta una declaración local, se consulta el nombre de la función actual (almacenado en la variable global `current_function`). El traductor genera un nombre único concatenando ambos términos (ej. `factorial_f`). Esta decisión permite mantener la simplicidad del código generado sin necesidad de implementar cierres (**closures**) complejos o estructuras `let` anidadas que habrían dificultado la generación de código lineal.

2.2. Estructuras de Control de Flujo

La traducción de estructuras de control requirió un análisis semántico de las diferencias entre ambos lenguajes:

- **Sentencias Condicionales:** En C, las ramas del `if` pueden contener múltiples sentencias. Dado que el `if` de Lisp solo acepta una expresión de «test», una de «then» y una de «else», se encapsularon las listas de sentencias dentro de bloques (`progn . . .`). Esto garantiza que todas las instrucciones se ejecuten secuencialmente y que el bloque devuelva el valor de la última expresión.
- **Bucles Iterativos:** Los bucles `while` y `for` se unificaron mediante la macro `loop`. Para el caso del `for`, la gramática extrae la inicialización y la coloca antes del bucle, mientras que la expresión de incremento se inserta automáticamente al final del bloque de sentencias, emulando con exactitud el comportamiento del estándar C99.

3. Resolución de Conflictos y Errores Sintácticos

Durante la fase de pruebas con el archivo `prueba.txt`, nos enfrentamos a errores críticos que obligaron a refinar la gramática en `trad2.y`:

3.1. El problema de la Definición de Parámetros

Originalmente, la gramática esperaba identificadores directos en la declaración de funciones. Al encontrar `suma(int x, int y)`, el compilador lanzaba un error de sintaxis en el token `int`. La solución consistió en redefinir la regla `lista_parametros` para que sea capaz de consumir el token `INTEGER` (el tipo de dato) pero solo procesar y almacenar el identificador. Esto permite que el programador escriba código C estándar mientras el traductor se centra únicamente en la estructura necesaria para Lisp.

3.2. Semántica del Switch y Omisión del Break

Un punto de debate fue la implementación del break. En C, el switch tiene un comportamiento de «caída» (**fall-through**). Sin embargo, el case de Lisp es inherentemente excluyente. Tras analizar los requisitos, determinamos que incluir el break en la gramática añadía una complejidad innecesaria de control de saltos. Decidimos que el traductor ignorara el token BREAK en la fase de análisis léxico o lo tratara como una sentencia vacía. Esto resulta en un código Lisp más limpio y funcional que mantiene la lógica esperada por el usuario original de C.

4. Implementación Técnica en Yacc/Bison

El archivo trad2.y utiliza una estructura jerárquica para la propagación de código. Cada regla sintáctica sintetiza un atributo .code que contiene la cadena de texto traducida. Hemos utilizado un buffer de salida global (output_buffer) y funciones auxiliares como emit() para organizar la generación de código, asegurando que las definiciones de variables globales siempre precedan a las funciones, tal como requiere el orden jerárquico de Common Lisp.

5. Conclusiones

La realización de este traductor ha permitido comprender profundamente las etapas del análisis sintáctico y la importancia de una gramática bien estructurada. La adaptación de C a Lisp demuestra que, a pesar de las diferencias superficiales de sintaxis, los conceptos de computación (asignación, iteración, recursividad) son universales, y que la clave de un buen traductor reside en la correcta gestión de la tabla de símbolos y el ámbito de las variables.